

On Deny Capabilities for Safe, Fast Actors

Sean T. Allen

Member of the **Pony** core team

[@SeanTAllen](#)

www.seantallen.com

This is a talk about **concurrency**.

This is a talk about **going fast**.

This talk is about **Pony** and **Deny Capabilities**.

About the paper

Actor model

Actor model basics

Actor model basics

- Actors communicate with other actors via messaging

Actor model basics

- Actors communicate with other actors via messaging
- Actors process messages

Actor model basics

- Actors communicate with other actors via messaging
- Actors process messages
- Actors "protect resources"

Safe

Data race freedom

Data race

Two memory accesses attempts where both:

Data race

Two memory accesses attempts where both:

- Target the same location

Data race

Two memory accesses attempts where both:

- Target the same location
- Are performed concurrently by two threads

Data race

Two memory accesses attempts where both:

- Target the same location
- Are performed concurrently by two threads
- Are not reads

Data race

Two memory accesses attempts where both:

- Target the same location
- Are performed concurrently by two threads
- Are not reads
- Are not synchronization operations

The actor model can help prevent data races

The actor model can help prevent data races

- All variables are "protected" by an actor - no global variables

The actor model can help prevent data races

- All variables are "protected" by an actor - no global variables
- Actors are processed sequentially 1 message a time by a single thread

The actor model can help prevent data races

- All variables are "protected" by an actor - no global variables
- Actors are processed sequentially 1 message a time by a single thread

Actors are "synchronization operations"

The actor model can help prevent data races

- All variables are "protected" by an actor - no global variables
- Actors are processed sequentially 1 message a time by a single thread

But sending data from one actor to another can cause problems...

Fast

How to go fast

How to go fast

- Avoid coordination

How to go fast

- Avoid coordination
- Avoid contention

How to go fast

- Avoid coordination
- Avoid contention
- Measure it

How actors can help with fast

How actors can help with fast

- make coordination explicit

How actors can help with fast

- make coordination explicit
- make contention explicit

How actors can hurt fast

How actors can hurt fast

- naive implementations can be very slow

How actors can hurt fast

- naive implementations can be very slow
- message queues are points of contention

How actors can hurt fast

- naive implementations can be very slow
- message queues are points of contention
- locks are usually faster than a large memory copy

Deny capabilities for **safe, fast** actors

Deny capabilities

doing unsafe fast things safely

Deny capabilities

statically confirm you aren't doing something
unsafe

Alias control

What is an alias?

What is an alias?

- Aliases are "names" for things in memory

What is an alias?

- Aliases are "names" for things in memory
- Aliases allow you to access a thing at a location

Aliases in Pony

Aliases in Pony

- When you **assign** a value to a variable or a field.

Aliases in Pony

- When you **assign** a value to a variable or a field.
- When you **pass** a value as an argument to a method.

Aliases in Pony

- When you **assign** a value to a variable or a field.
- When you **pass** a value as an argument to a method.
- When you **call a method**, an alias of the receiver of the call is created. It is accessible as **this** within the method body.

Count the aliases

```
primitive Say
  fun say(msg: String, out: OutStream) =>
    let x = "We say '" + msg + "'"
    out.print(x)
```

Count the aliases

primitive Say

```
fun say(msg: String, out: OutputStream) =>  
    let x = "We say '" + msg + "'"   
    out.print(x)
```

msg and **out** are aliases

Count the aliases

```
primitive Say
fun say(msg: String, out: OutputStream) =>
    let x = "We say '" + msg + "'"
    out.print(x)
```

x is an alias

Count the aliases

```
primitive Say
fun say(msg: String, out: OutputStream) =>
    let x = "We say '" + msg + "'"
    out.print(x)
```

x is aliased when passed to **print**

Count the aliases

```
primitive Say
  fun say(msg: String, out: OutputStream) =>
    let x = "We say '" + msg + "'"
    out.print(x)
```

Capabilities

Annotations on code that can be used to statically confirm some property

```
actor Main
  new create(env: Env) =>
    let msg: String val = "Hello World!"
    env.out.print(msg)
```



```
actor Main
  new create(env: Env) =>
    let msg: String val = "Hello World!"
    env.out.print(msg)
```

val is a capability

```
actor Main
  new create(env: Env) =>
    let msg: String val = "Hello World!"
    env.out.print(msg)
```

val is part of the type at compile-time

```
actor Main
  new create(env: Env) =>
    let msg: String val = "Hello World!"
    env.out.print(msg)
```

val says "the alias **msg** to the String **Hello World** is immutable"

What can you deny?

What can you deny?

- Reading

What can you deny?

- Reading
- Mutating

What can you deny?

- Reading
- Mutating
- Aliasing

What can you deny?

- Reading
- Mutating
- Aliasing
- Sending

What can you deny?

- Reading
- Mutating
- Aliasing
- Sending
- Sharing

Some capabilities

Some capabilities

- ref

Some capabilities

- ref
- val

Some capabilities

- ref
- val
- iso

Some capabilities

- ref
- val
- iso
- tag

ref

ref

- **allows** reading

ref

- **allows** reading
- **allows** mutation

ref

- **allows** reading
- **allows** mutation
- **denies** sending

ref

- **allows** reading
- **allows** mutation
- **denies** sending
- **allows** unlimited aliases

ref

- **allows** reading
- **allows** mutation
- **denies** sending
- **allows** unlimited aliases
- **denies** sharing

iso

iso

- **allows** reading

iso

- **allows** reading
- **allows** mutation

iso

- **allows** reading
- **allows** mutation
- **allows** sending

iso

- **allows** reading
- **allows** mutation
- **allows** sending
- **denies** aliasing

iso

- **allows** reading
- **allows** mutation
- **allows** sending
- **denies** aliasing
- **denies** sharing

val

val

- **allows** reading

val

- **allows** reading
- **denies** mutation

val

- **allows** reading
- **denies** mutation
- **allows** sending

val

- **allows** reading
- **denies** mutation
- **allows** sending
- **allows** unlimited aliases

val

- **allows** reading
- **denies** mutation
- **allows** sending
- **allows** unlimited aliases
- **allows** sharing

tag

tag

- **denies** reading

tag

- **denies** reading
- **denies** mutation

tag

- **denies** reading
- **denies** mutation
- **allows** aliasing

tag

- **denies** reading
- **denies** mutation
- **allows** aliasing
- **allows** sharing

tag

- **denies** reading
- **denies** mutation
- **allows** aliasing
- **allows** sharing
- **allows** sending

Readable

Readable

- ref

Readable

- ref
- iso

Readable

- ref
- iso
- val

Readable

- ref
- iso
- val

Mutable

Readable

- ref
- iso
- val

Mutable

- ref

Readable

- ref
- iso
- val

Mutable

- ref
- iso

Readable

- ref
- iso
- val

Mutable

- ref
- iso

Aliasable

Readable

- ref
- iso
- val

Mutable

- ref
- iso

Aliasable

- ref

Readable

- ref
- iso
- val

Mutable

- ref
- iso

Aliasable

- ref
- val

Readable

- ref
- iso
- val

Mutable

- ref
- iso

Aliasable

- ref
- val
- tag

Readable

- ref
- iso
- val

Mutable

- ref
- iso

Sendable

Aliasable

- ref
- val
- tag

Readable

- ref
- iso
- val

Mutable

- ref
- iso

Sendable

- val

Aliasable

- ref
- val
- tag

Readable

- ref
- iso
- val

Mutable

- ref
- iso

Sendable

- val
- tag

Aliasable

- ref
- val
- tag

Readable

- ref
- iso
- val

Mutable

- ref
- iso

Sendable

- val
- tag
- iso

Aliasable

- ref
- val
- tag

Readable

- ref
- iso
- val

Mutable

- ref
- iso

Shareable

Sendable

- val
- tag
- iso

Aliasable

- ref
- val
- tag

Readable

- ref
- iso
- val

Mutable

- ref
- iso

Shareable

- val

Sendable

- val
- tag
- iso

Aliasable

- ref
- val
- tag

Readable

- ref
- iso
- val

Mutable

- ref
- iso

Shareable

- val
- tag

Sendable

- val
- tag
- iso

Aliasable

- ref
- val
- tag

There's no platonic ideal for deny capabilities.

Code (and errors!)

```
actor Main
  new create(env: Env) =>
    let msg: String ref = "Hello World!".clone()
    env.out.print(msg)
```



```
actor Main
  new create(env: Env) =>
    let msg: String ref = "Hello World!".clone()
    env.out.print(msg)
```

Hello World String is mutable

```
actor Main
  new create(env: Env) =>
    let msg: String ref = "Hello World!".clone()
    env.out.print(msg)
```

send **msg** to actor **out**

```
actor Main
  new create(env: Env) =>
    let msg: String ref = "Hello World!".clone()
    env.out.print(msg)
```

ref isn't sendable

```
actor Main
  new create(env: Env) =>
    let msg: String ref = "Hello World!".clone()
    env.out.print(msg)
```

which will result in an error here

Error:

```
main.pony:4:19: argument not a subtype of parameter
  env.out.print(msg)
                ^
```

Info:

```
main.pony:4:19: argument type is String ref
  env.out.print(msg)
                ^
```

```
(...):13:12: parameter type is
  (String val | Array[U8 val] val)
    be print(data: ByteSeq)
                ^
```

```
main.pony:3:14: String ref is not a subtype of String val:
  ref is not a subcap of val
    let msg: String ref = "Hello World!".clone()
                ^
```

```
main.pony:3:14: String ref is not a subtype of Array[U8 val] val
```


Error:

```
main.pony:4:19: argument not a subtype of paramete  
    env.out.print(msg)
```

^

Info:

```
main.pony:4:19: argument type is String ref  
    env.out.print(msg)
```

^

```
(...):13:12: parameter type is  
    (String val | Array[U8 val] val)  
    be print(data: ByteSeq)
```

^

```
main.pony:3:14: String ref is not a subtype of String val:  
    ref is not a subcap of val  
    let msg: String ref = "Hello World!".clone()
```

^

msg isn't the correct type

Error:

```
main.pony:4:19: argument not a subtype of parameter
  env.out.print(msg)
                ^
```

Info:

```
main.pony:4:19: argument type is String ref
env.out.print(msg)
                ^
```

```
(...):13:12: parameter type is
  (String val | Array[U8 val] val)
    be print(data: ByteSeq)
                ^
```

```
main.pony:3:14: String ref is not a subtype of String val:
  ref is not a subcap of val
    let msg: String ref = "Hello World!".clone()
                ^
```

msg is String ref

```
env.out.print(msg)
```

^

Info:

```
main.pony:4:19: argument type is String ref
```

```
env.out.print(msg)
```

^

```
(...):13:12: parameter type is  
  (String val | Array[U8 val] val)  
  be print(data: ByteSeq)
```

^

```
main.pony:3:14: String ref is not a subtype of String val:
```

```
ref is not a subcap of val
```

```
let msg: String ref = "Hello World!".clone()
```

^

```
main.pony:3:14: String ref is not a subtype of Array[U8 val] val
```

print requires **String val** or **Array[U8] val**

The compiler just statically checked our data sharing.

The compiler just statically checked our data sharing.
Let's fix it

```
actor Main
  new create(env: Env) =>
    let msg: String val = "Hello World!"
    env.out.print(msg)
```

```
actor Main
  new create(env: Env) =>
    let msg: String val = "Hello World!"
    env.out.print(msg)
```

Hello World String is immutable

```
actor Main
  new create(env: Env) =>
    let msg: String val = "Hello World!"
    env.out.print(msg)
```

send **msg** to actor **out**


```
actor Main
  new create(env: Env) =>
    let msg: String val = "Hello World!"
    env.out.print(msg)
```

val is sendable

```
actor Main
  new create(env: Env) =>
    let msg: String val = "Hello World!"
    env.out.print(msg)
```

no error here

Learn more

<https://www.seantallen.com/talks/deny-capabilities/>